Xi Zhang

+86 18621813302 | flashzxi@outlook.com

Education

Fudan University 2021 – 2024
M.S. in Computer Science and Technology
Research focus: Database Systems and High-Performance Systems.
Paper: *SDEcho: Efficient Explanation of Aggregated Sequence Difference* (VLDB), second author.

Shanghai Jiao Tong University 2017 – 2021
B.S. in Physics
Outstanding Graduate of Shanghai Jiao Tong University (2021).

Experience

Baidu – MEG Technology Platform, BaikalDB Team Mar 2024 – Present
R&D Engineer

- Working on database kernel and infrastructure development across query optimization, execution engines, index and storage systems, cross-architecture adaptation, and performance engineering.
- Led and implemented optimizations in core execution paths including joins, subquery rewriting, inverted indexes, log storage, and ARM migration, delivering stable production performance gains.

Microsoft – Microsoft 365 Cloud PC Business Sep 2023 – Jan 2024
R&D Engineer Intern

- Contributed to the development and optimization of Cloud PC resource allocation services.

Selected Project Experience

Columnar Join Optimization for LargeBinary Keys *C++ / SIMD / Apache Arrow*

- Designed and implemented a dedicated high-performance join path for LargeBinary string join scenarios, addressing the reuse bottleneck of Swiss Join.
- Reworked the in-set string representation to `uint64_t`, redesigned SwissTable memory layout, and replaced string-aligned copying with BinaryView to reduce memory overhead.
- Applied SIMD vectorization to critical encoding, decoding, and string-comparison paths.
- **Impact:** For string join keys, join memory overhead decreased by **50%**; CPU overhead decreased by **45%–70%**, with an additional improvement of about **20%** when SIMD was enabled.

Query Optimizer and Inverted Index Performance Optimization *C++ / Query Optimizer*

- Derived and implemented equivalence rewrite rules for `ANY/ALL` subqueries to `EXISTS/NOT EXISTS`, converting more plans into semi-join and anti-semi-join execution and avoiding expensive nested loops.
- Identified inverted-index hotspots via CPU flame-graph analysis, refined recursive progression logic, and improved covering-index evaluation through earlier termination.
- Optimized post-hit scan-range clipping for secondary indexes by leveraging auto-completed primary-key columns, significantly reducing invalid scan overhead.
- Implemented core CBO statistics components based on PostgreSQL-style upgrade statistics, including HyperLogLog, MCV, histograms, and secondary-index support for NULL columns.
- **Impact:** Typical inverted-index query latency improved from **4.406 s** to **0.563 s** (**6.8x** faster). Rewritten nested subquery complexity improved from $O(mN)$ to $O(m \log n)$. SQL statements requiring `force index` were reduced by about **90%**.

Infrastructure and Cross-Architecture Performance Optimization *C++ / RocksDB / ARM*

- Designed and implemented TSO request-packet aggregation within a **3 ms** time window to reduce metadata-service pressure in large clusters.
- Developed a new `raft_log_storage` based on FIFO compaction with expiration hooks to avoid clearing valid logs and compacting expired logs unnecessarily.
- Adapted BaikalDB, BaikalMeta, and related dependencies for ARM, resolving architecture-specific issues across RocksDB, brpc, gperftools, and the ARM test pipeline.
- **Impact:** FIFO raft-log write speed improved by **43%–86%** in high-QPS write scenarios. On ARM compared with x86, select throughput increased by **70.5%**, latency decreased by **55%**, and update throughput increased by **52%**.

Tooling and Data Lifecycle Features *C++*

- Built a MySQL-to-BaikalDB import tool that migrates existing MySQL data through simple account and table configuration.
- Implemented specified-column TTL and extended TTL support to automatically delete data in tables with inverted or vector indexes.
- Developed DDL DELETE functionality based on online DDL logic to safely bypass transactions during very large deletions and protect cluster stability.
- **Impact:** For a typical ultra-large table with **4 billion** records, import time dropped from about **5 days** to **1 day**, lowering migration cost and improving user-side data operations.

IoTDB Expression Code Generation (Open Source Summer 2022) *Java / Query Execution Engine*

- Contributed to Apache IoTDB expression code generation for more efficient expression evaluation under the Volcano model.
- Designed a code-generation solution that converts expression trees and DAGs into dynamically compiled flattened script code, reducing overhead from deep virtual-call chains, repeated type checks, and redundant sub-expression evaluation.
- Supported arithmetic expressions, logical expressions, and partially mappable UDFs, laying the groundwork for follow-up optimization of scan, filter, and transform operators.

InfiniTensor Large Model and AI System Training Camp (Winter 2025, Outstanding Participant) *CUDA / C++ / Python*

- Optimized CUDA kernels for operators including FlashAttention v2, NF4 Decode, and Hadamard Transform, covering kernel design, memory-access optimization, and performance tuning.
- Improved performance through thread-block partitioning, shared-memory usage, memory-access consolidation, vectorized loading, and instruction-level parallelism, followed by correctness validation and benchmarking.
- Used `pybind11` to build conversion capability from PyTorch ATen graphs to backend computational graphs, establishing a mapping chain from frontend operators to backend IR.

Skills

Programming: C++, Python, CUDA, Java
Systems & Performance: Linux, SIMD, zero-copy, flame-graph analysis, memory-access optimization
Core Areas: Query optimization, execution engines, rule rewriting, cost-based optimization, storage and infrastructure performance